Sales Forecasting & Demand Prediction

DEPI’s Final Project

Supervisor:
Eng. Laila Ahmed

SPECIAL GREETINGS TO:

Dr. Laila Ahmed

Agenda

01 Project overview & goals

02 Data collection, EDA, preprocessing

03 Time series analysis & feature engineering

04 Model training and evaluation

05 Model deployment

06 Conclusion

Team members

MERIT MARY SHAFEEQ

MOHAMED ALI

SAMA A.ELMOGY

MOHAMED AYMAN

MONTAHA AHMED

Project Workflow Sequence

Data Collection & Cleaning

- Collect data from CSV.
- Remove duplicates, handle missing values.
- Detect & treat outliers.

EDA & Feature Engineering

- Analyze data distributions and create visualizations.
- Extract key insights.
- Create additional features.

Model Selection & Training

- Test multiple models: Linear Regression, Random Forest, XGBoost, etc.
- Evaluate performance on Train/Test sets.

Hyperparameter Tuning & Model Evaluation

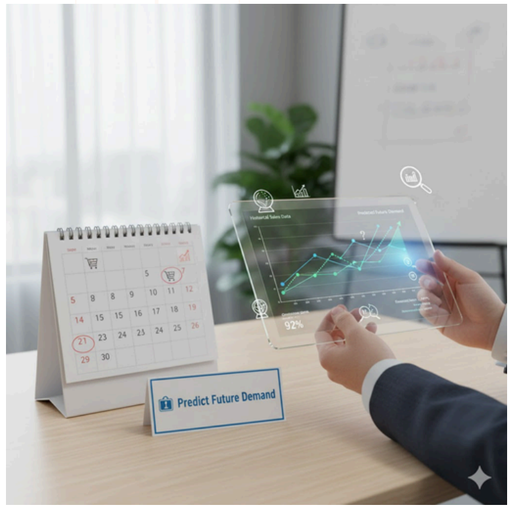
- Tune parameters using GridSearchCV.
- Calculate tuned metrics: RMSE, MAE, R^2.

Insights, Recommend. & Reporting

- Extract key trends and patterns from the model.
- Prepare dashboard or final report.

Project Objectives& Problem Statement

The problem addressed in this project is the difficulty of predicting continuous numerical outcomes based on historical data. Without a proper predictive model, these predictions can be inaccurate and unreliable.
This project focuses on building a regression model that can understand patterns in the data and generate accurate predictions.



Import libaries and setup

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import plotly.express as px
import warnings
warnings.filterwarnings("ignore")
import plotly.graph_objects as go
```

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, classification_report
import xgboost as xgb
import pickle
```

Dataset

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	City	State	...	Product ID	Category	Sub-Category	Product Name	Sales	Quantity	Discount	Profit	Shipping Cost	Order Priority
0	32298	CA-2012-124891	2012-07-31	31-07-2012	Same Day	RH-19495	Rick Hansen	Consumer	New York City	New York	...	TEC-AC-10003033	Technology	Accessories	Plantronics CS510 - Over-the-Head monaural Wir...	2309.650	7	0.0	762.1845	933.57	Critical
1	26341	IN-2013-77878	2013-02-05	07-02-2013	Second Class	JR-16210	Justin Ritter	Corporate	Wollongong	New South Wales	...	FUR-CH-10003950	Furniture	Chairs	Novimex Executive Leather Armchair, Black	3709.395	9	0.1	-288.7650	923.63	Critical
2	25330	IN-2013-71249	2013-10-17	18-10-2013	First Class	CR-12730	Craig Reiter	Consumer	Brisbane	Queensland	...	TEC-PH-10004664	Technology	Phones	Nokia Smart Phone, with Caller ID	5175.171	9	0.1	919.9710	915.49	Medium
3	13524	ES-2013-1579342	2013-01-28	30-01-2013	First Class	KM-16375	Katherine Murray	Home Office	Berlin	Berlin	...	TEC-PH-10004583	Technology	Phones	Motorola Smart Phone, Cordless	2892.510	5	0.1	-96.5400	910.16	Medium
4	47221	SG-2013-4320	2013-11-05	06-11-2013	Same Day	RH-9495	Rick Hansen	Consumer	Dakar	Dakar	...	TEC-SHA-10000501	Technology	Copiers	Sharp Wireless Fax, High-Speed	2832.960	8	0.0	311.5200	903.04	Critical

5 rows × 24 columns

The dataset used in this project is the Global Superstore Sales Dataset from Kaggle, with around 50,000 rows and 24 columns. It includes essential details about orders, customers, markets, products, and key sales metrics such as sales, profit, discount, and shipping cost.
This dataset offers a strong base for analyzing sales trends and developing forecasting models.

Dataset

Info about the dataset:

	Row ID	Postal Code	Sales	Quantity	Discount	Profit	Shipping Cost
count	51290.00000	9994.000000	51290.000000	51290.000000	51290.000000	51290.000000	51290.000000
mean	25645.50000	55190.379428	246.490581	3.476545	0.142908	28.610982	26.375915
std	14806.29199	32063.693350	487.565361	2.278766	0.212280	174.340972	57.296804
min	1.00000	1040.000000	0.444000	1.000000	0.000000	-6599.978000	0.000000
25%	12823.25000	23223.000000	30.758625	2.000000	0.000000	0.000000	2.610000
50%	25645.50000	56430.500000	85.053000	3.000000	0.000000	9.240000	7.790000
75%	38467.75000	90008.000000	251.053200	5.000000	0.200000	36.810000	24.450000
max	51290.00000	99301.000000	22638.480000	14.000000	0.850000	8399.976000	933.570000

```
num_records = df.shape[0]
print(f"> Number of records in the dataset: {num_records}")
```

```
> Number of records in the dataset: 51290
```

```
total_sales = df['Sales'].sum()
print(f"> Total Sales Revenue: ${total_sales:,.2f}")
```

```
> Total Sales Revenue: $12,642,501.91
```

```
total_profit = df['Profit'].sum()
print(f"> Total Profit: ${total_profit:,.2f}")
```

```
> Total Profit: $1,467,457.29
```

```
total_discounts = df['Discount'].sum()
print(f"> Total Discounts Given: {total_discounts:.2f}")
```

```
> Total Discounts Given: 7329.73
```

Data Cleaning

Cleaning Actions:

- Removed duplicate rows
- Converted numeric columns (Sales, Quantity, Discount, Profit, Shipping Cost)
- Fixed missing values (numeric → filled with 0 after coercion)
- Standardized date formats (Order Date & Ship Date)
- Created clean time features (Year, Month, Day of Week, Weekend Flag)
- Ensured valid processing time (Ship Date - Order Date)
- Handled invalid values using safe type conversion (to_numeric with errors='coerce')

Data Cleaning

Code and information of data after cleaning :

```python
# --------- 1. Load and clean ----------
df = pd.read_csv("Original Data.csv", parse_dates=["Order Date","Ship Date"], dayfirst=True, encoding='latin1')

df.drop_duplicates(inplace=True)

for c in ["Sales","Quantity","Discount","Profit","Shipping Cost"]:
    df[c] = pd.to_numeric(df[c], errors="coerce").fillna(0)

# --------- 2. Time features ----------
df["Year"] = df["Order Date"].dt.year
df["Month"] = df["Order Date"].dt.month
df["Day_of_Week"] = df["Order Date"].dt.dayofweek
df["Is_Weekend"] = df["Day_of_Week"].isin([4,5,6]).astype(int)
df["Processing Time (Days)"] = (df["Ship Date"] - df["Order Date"]).dt.days.fillna(0).astype(int)
```

Data cleaning

Code and information of data after cleaning :

```
from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()

# Columns that Label Encoder will be applied to
categorical_columns = ["Order Priority", "Ship Mode", "Segment", "Market", "Category", "Sub-Category", "Region", "Season"]

# Applying Label Encoder
for col in categorical_columns:
    df[col + "_Encoded"] = encoder.fit_transform(df[col])

# Show Modified Columns
df[[*categorical_columns, *[col + "_Encoded" for col in categorical_columns]]].head()
```

Data Cleaning

Code and information of data after cleaning :

```
duplicates_values = df.duplicated().sum()
print(f"Number of Duplicates Values = {duplicates_values}")
```

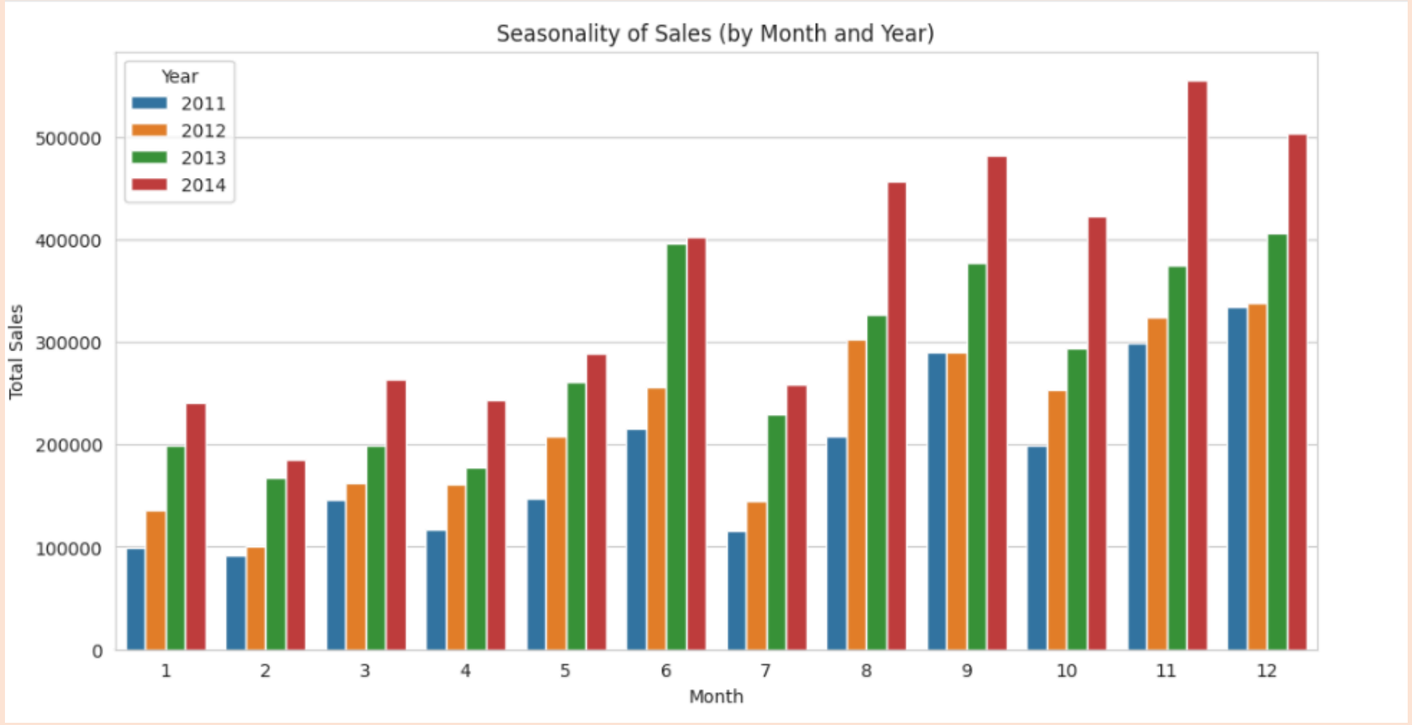
```
Number of Duplicates Values = 0
```

```
missing_values = pd.DataFrame(df.isna().sum())
missing_values
```

	0
Order Date	0
Ship Date	0
Ship Mode	0
Customer Name	0
Segment	0
City	0
State	0
Country	0
Market	0
Region	0
Category	0
Sub-Category	0
Product Name	0
Sales	0
Quantity	0
Discount	0
Profit	0
Shipping Cost	0
Order Priority	0
Month	0
Quarter	0
Year	0
Day_of_Week	0
Is_Weekend	0

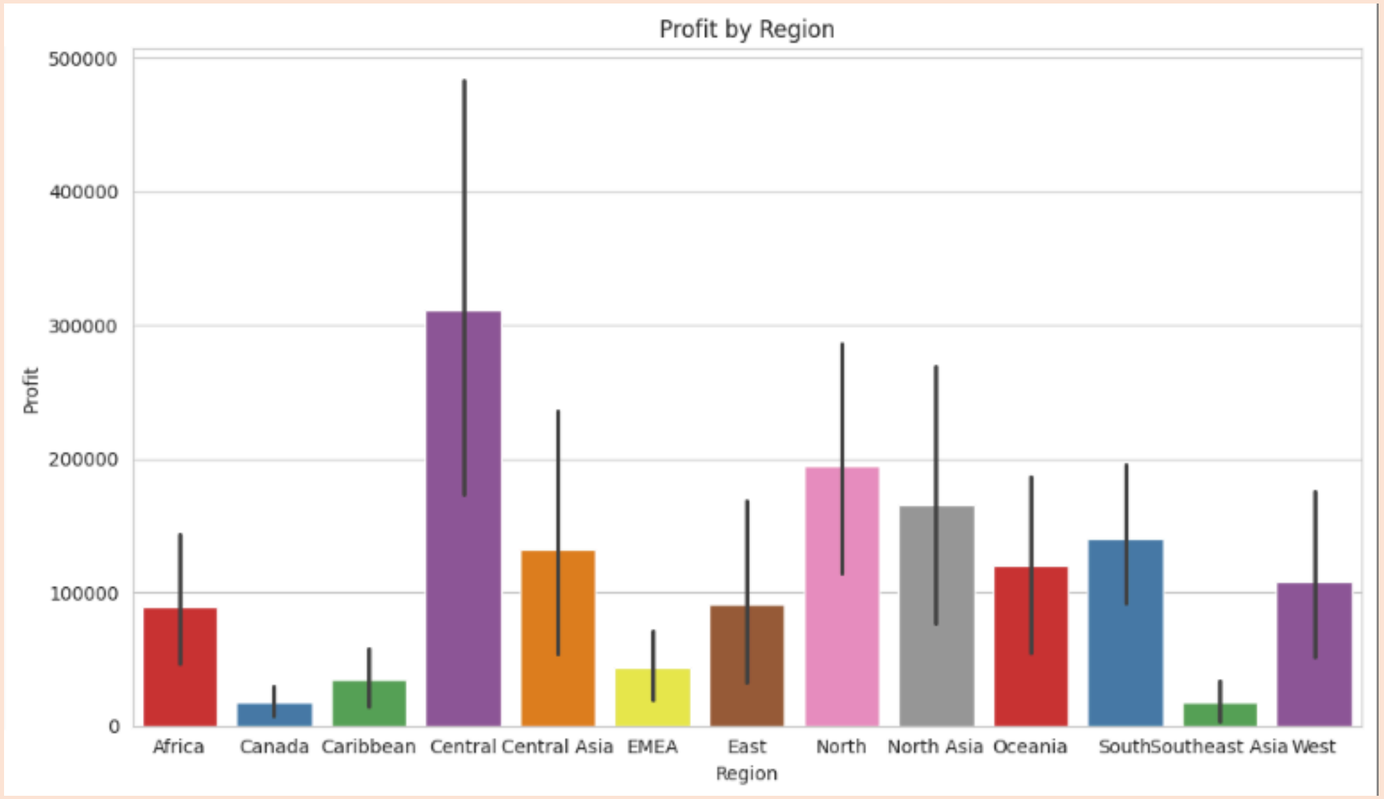
Exploratory Data Analysis

Time series: monthly sales trends visualization



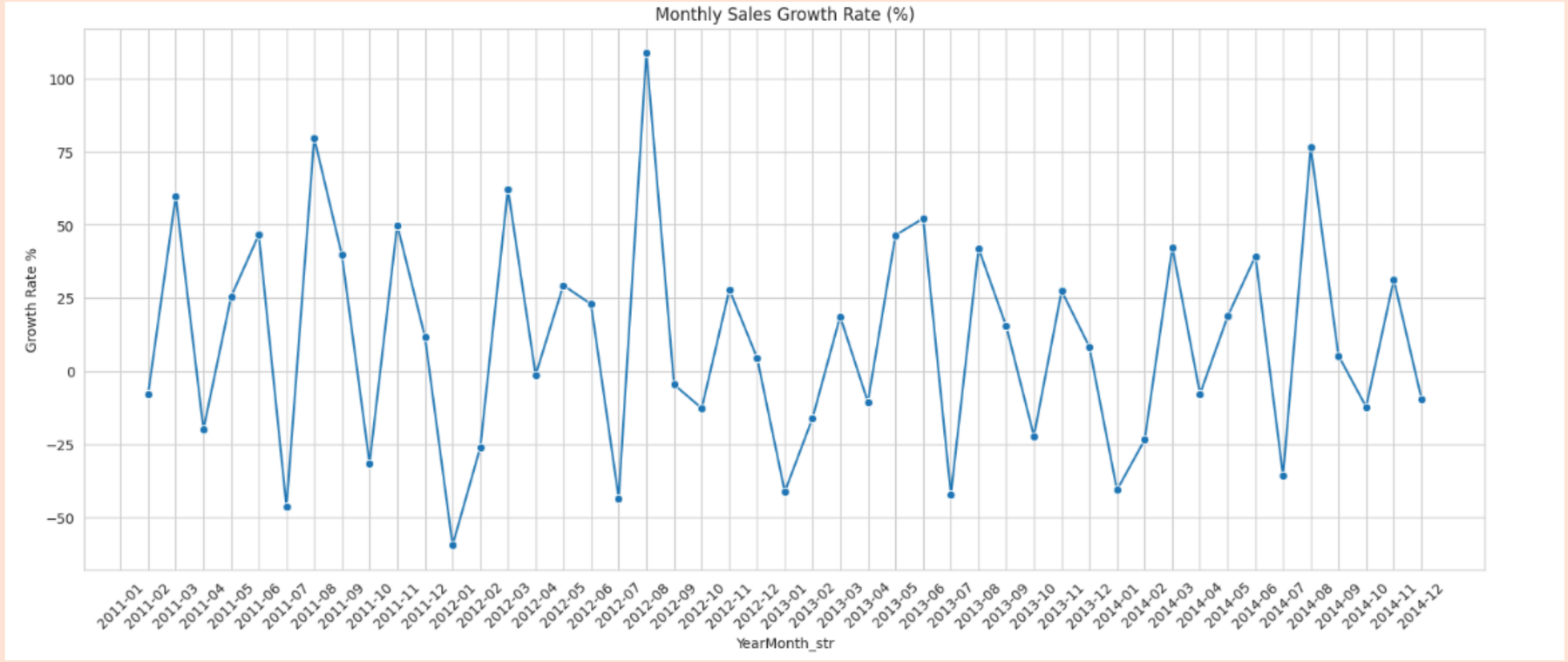
Exploratory Data Analysis

Profit by region boxplot



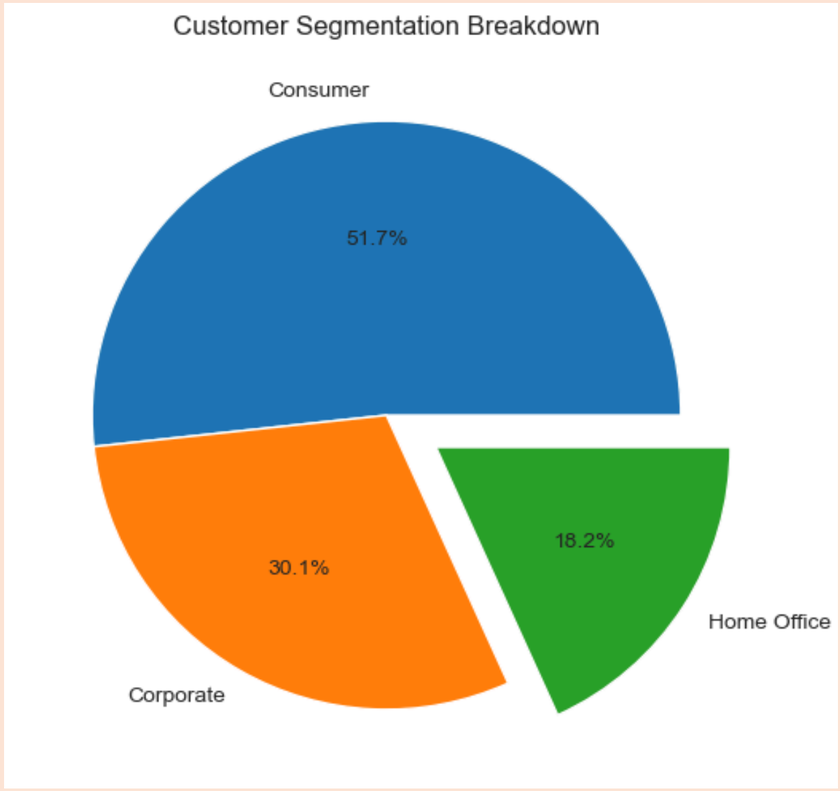
Exploratory Data Analysis

Monthly sales growth rate



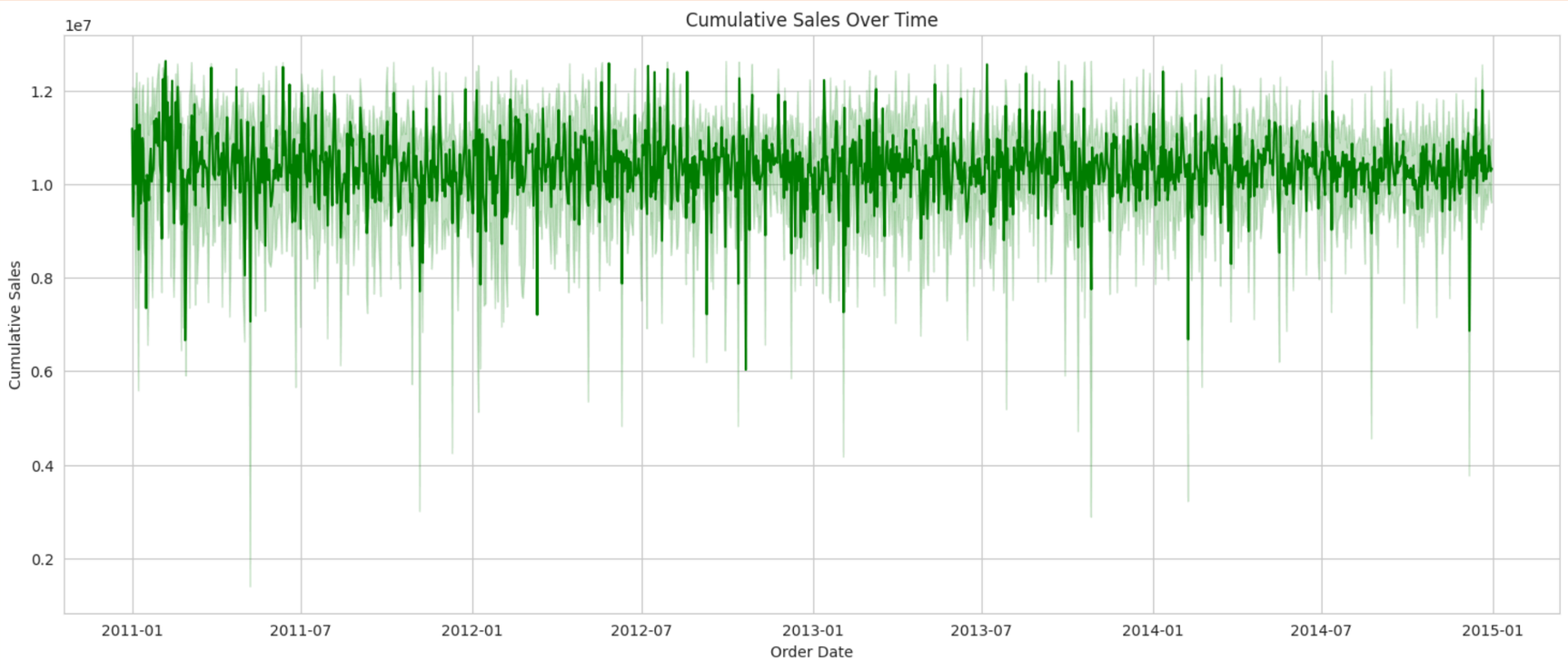
Exploratory Data Analysis

Customer segmentation breakdown



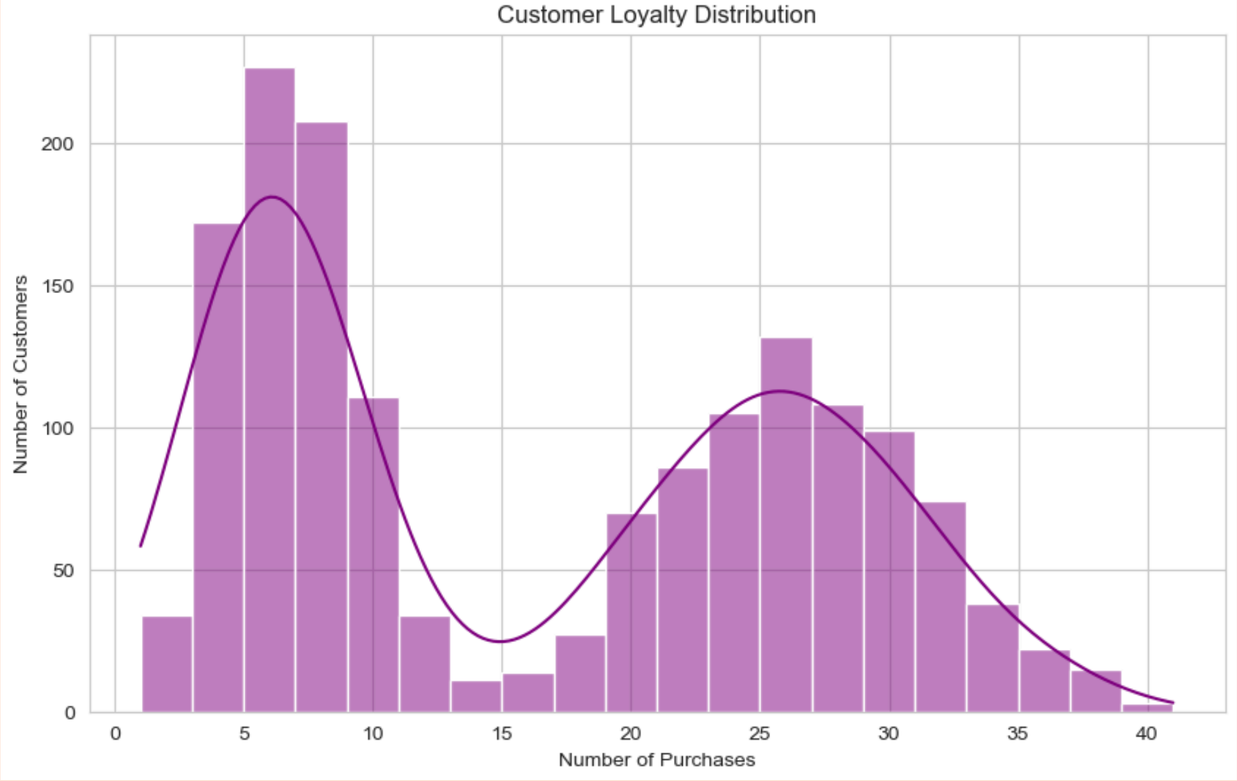
Exploratory Data Analysis

Cumulative sales over time



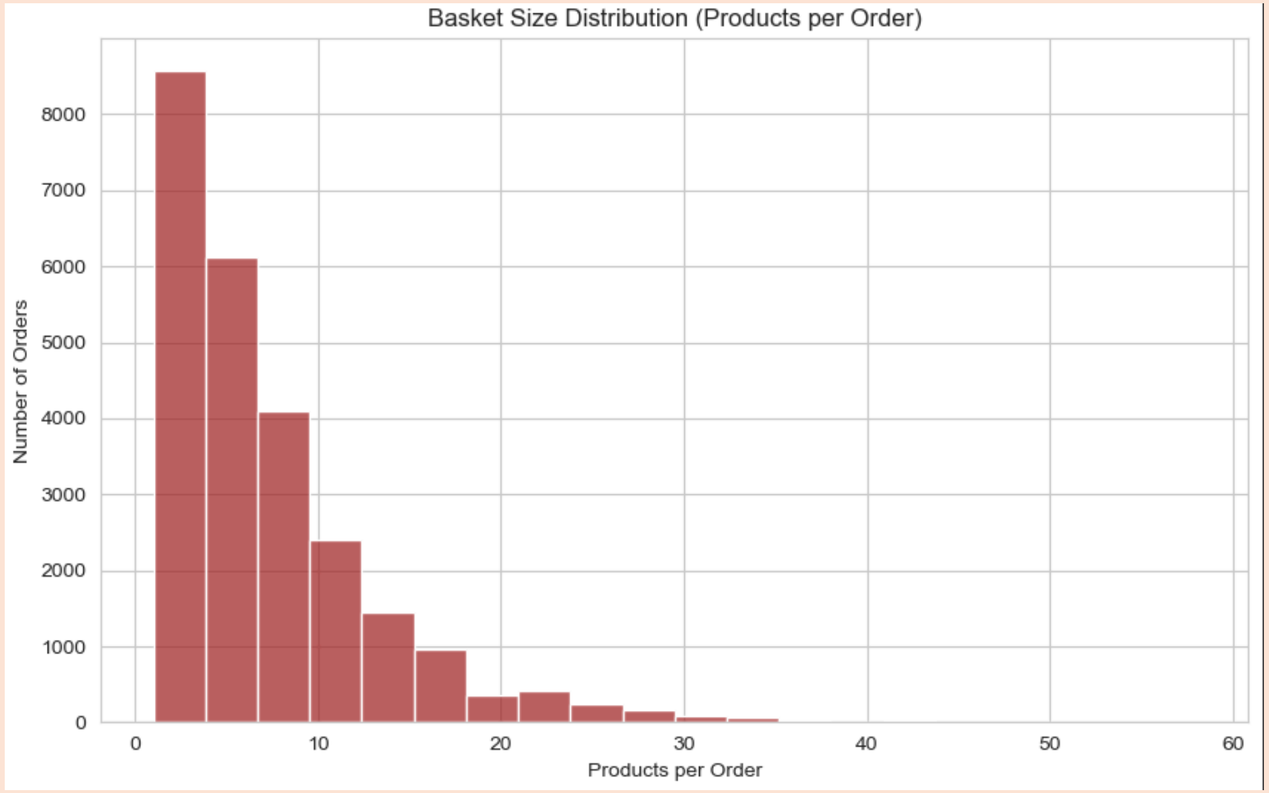
Exploratory Data Analysis

Customer loyalty distribution



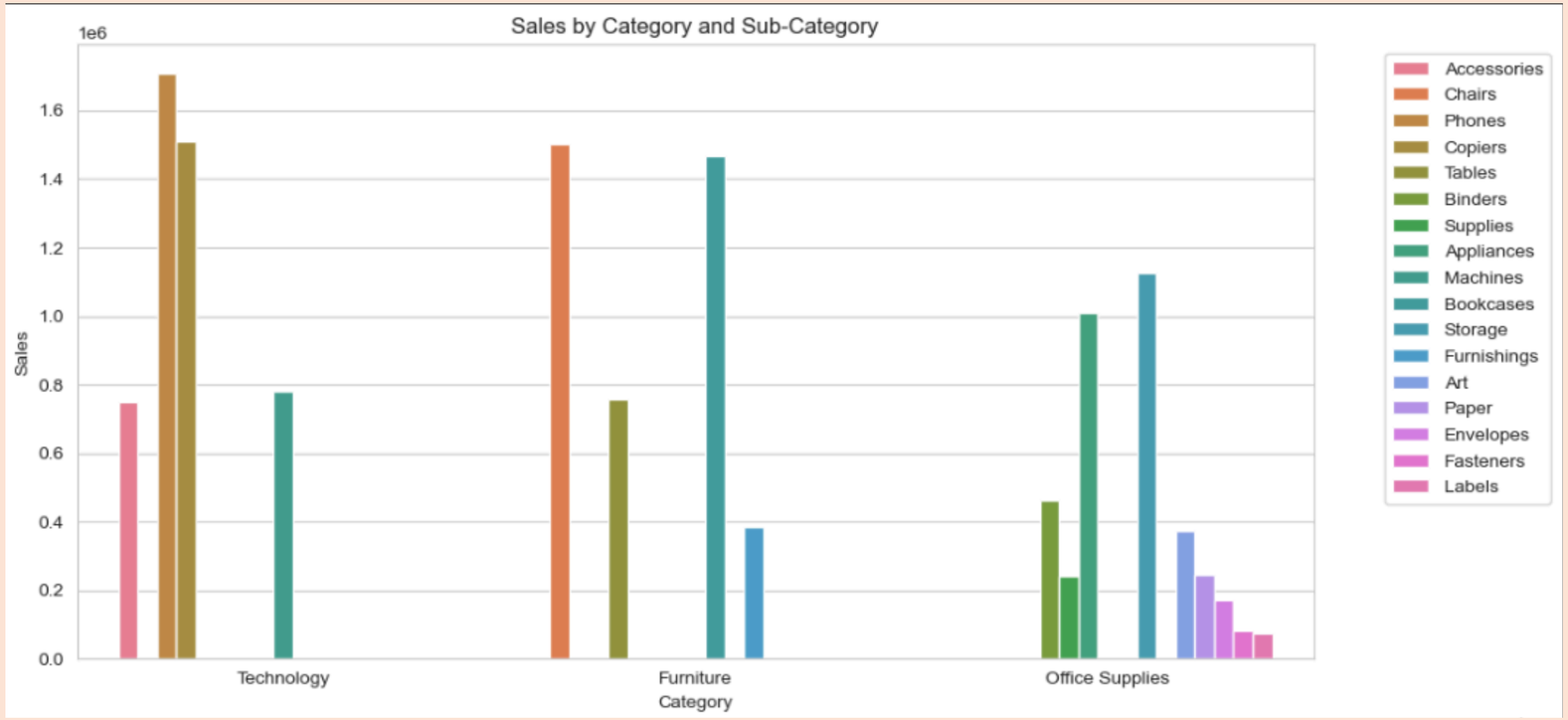
Exploratory Data Analysis

How many products customers usually buy per order



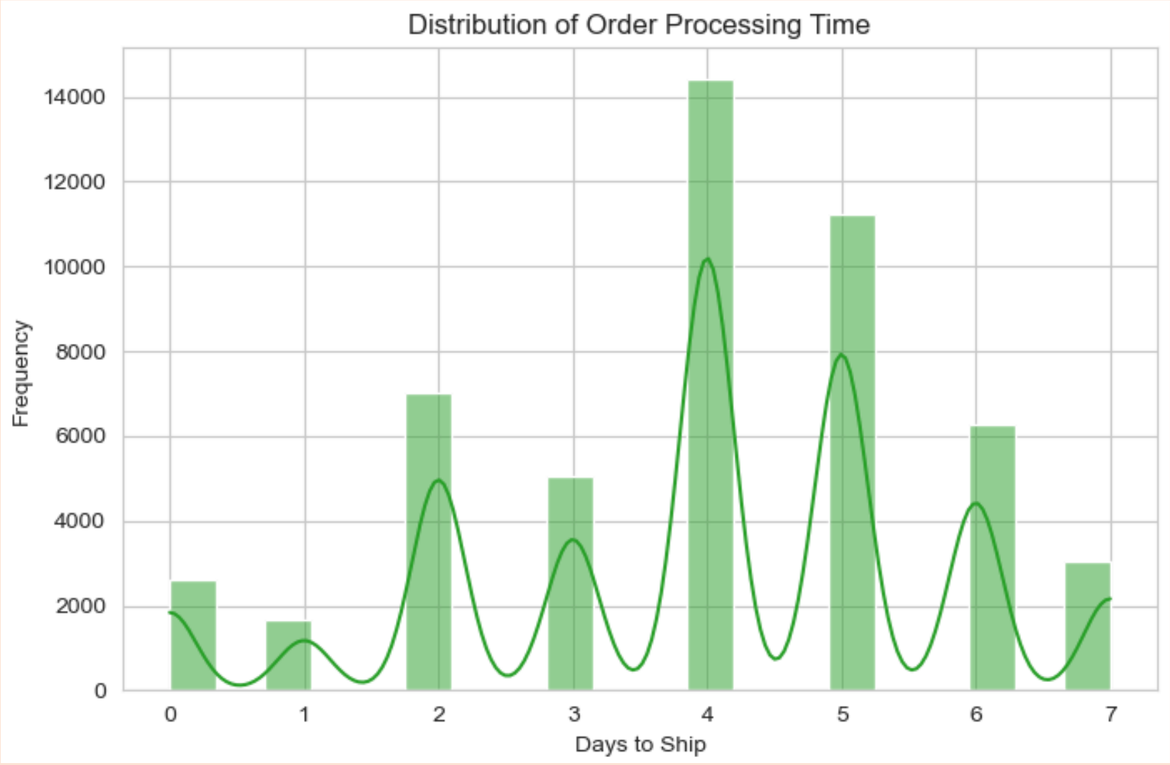
Exploratory Data Analysis

Sales by category & sub-category



Exploratory Data Analysis

Distribution of order processing time

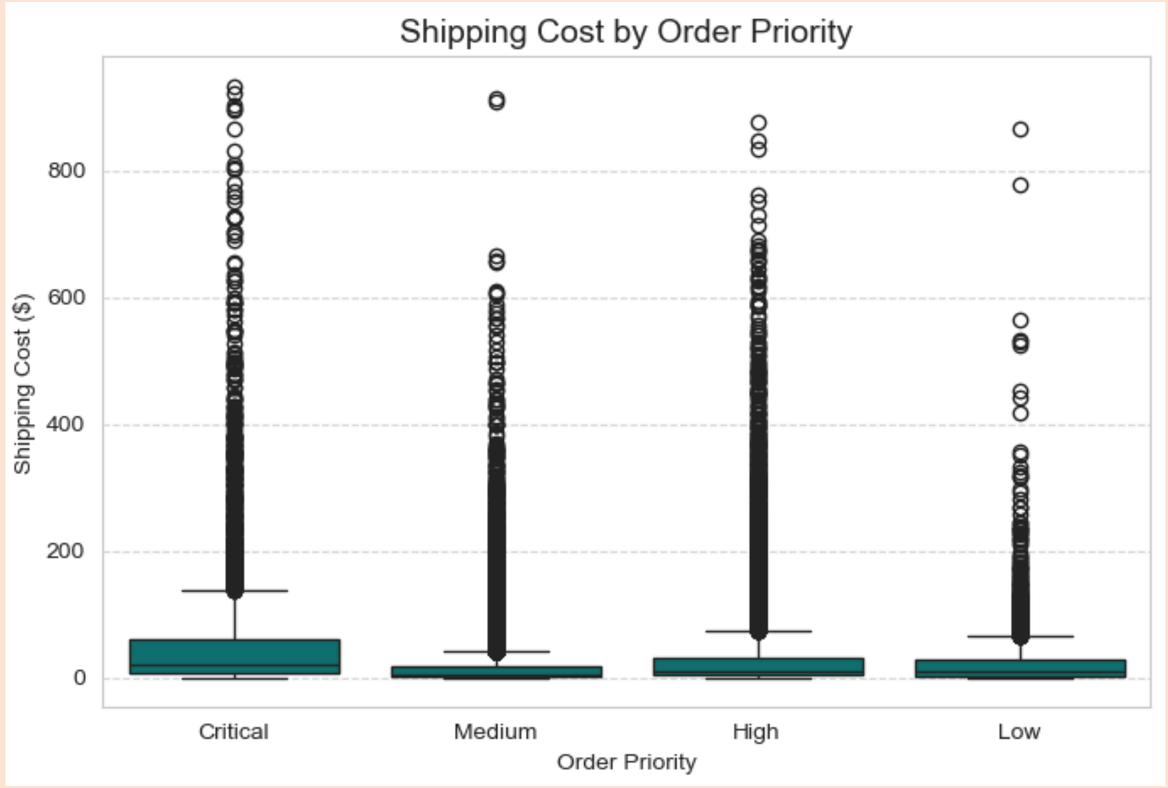


Exploratory Data Analysis

Distribution of shipping modes

Exploratory Data Analysis

Shipping cost by order priority



Feature engineering

Some of the features taht we added to the dataset :

- Time features: Month, Week, Day of Week, Is_Weekend
- Lag features: sales_lag_1, sales_lag_4 (previous week/month performance)
- Rolling features: rolling_mean_4w, rolling_std_4w (short-term trends)
- We dropped some columns that didn’t have that much of impact on the output like('Row ID' ,'Order ID' ,'Customer ID','Postal Code','Product ID')

Feature engineering

The dataset after performing feature engineering:

Order Date	Ship Date	Ship Mode	Customer Name	Segment	City	State	Country	Market	Region	...	stock_level	lead_time	stockout_risk	promo_uplift	ad_spend	holiday_proximity	economic_index	weather_index
2012-07-31	2012-07-31	Same Day	Rick Hansen	Consumer	New York City	New York	United States	US	East	...	35	4.3	0	0.0000	0.0	3	89.767133	0.759536
2013-02-05	2013-02-07	Second Class	Justin Ritter	Corporate	Wollongong	New South Wales	Australia	APAC	Oceania	...	37	3.6	0	2793.4950	10.0	2	94.327838	0.072096
2013-10-17	2013-10-18	First Class	Craig Reiter	Consumer	Brisbane	Queensland	Australia	APAC	Oceania	...	52	2.8	0	3460.7625	10.0	0	82.089321	0.845035
2013-01-28	2013-01-30	First Class	Katherine Murray	Home Office	Berlin	Berlin	Germany	EU	Central	...	19	2.8	0	1606.9500	10.0	3	100.619064	0.667697
2013-11-05	2013-11-06	Same Day	Rick Hansen	Consumer	Dakar	Dakar	Senegal	Africa	Africa	...	8	1.0	0	0.0000	0.0	9	102.674214	0.584166

ws × 45 columns

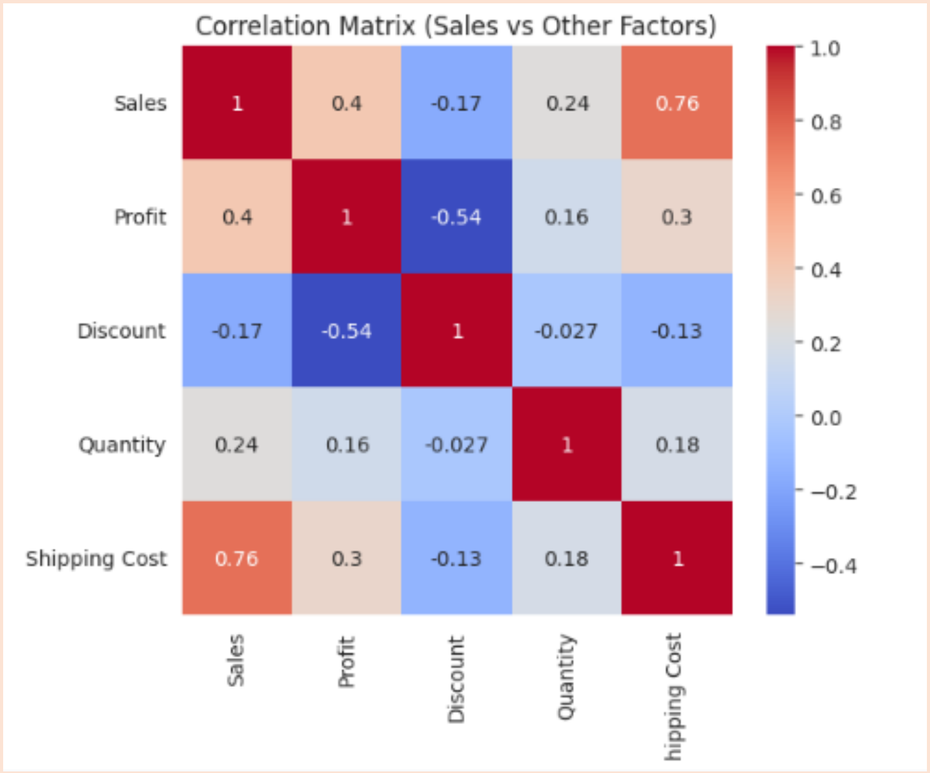
Data Analysis & Visualization

what we did in the data analysis and visualization part is that we :

- Explored relationships between sales, profit, discount, and quantity
- Identified long term sales trend
- Evaluated impact of promotions and discounts on sales
- Analyzed long-term trends using rolling averages
- Examined profit vs. shipping cost relationships
- Created interactive visualizations for dashboards

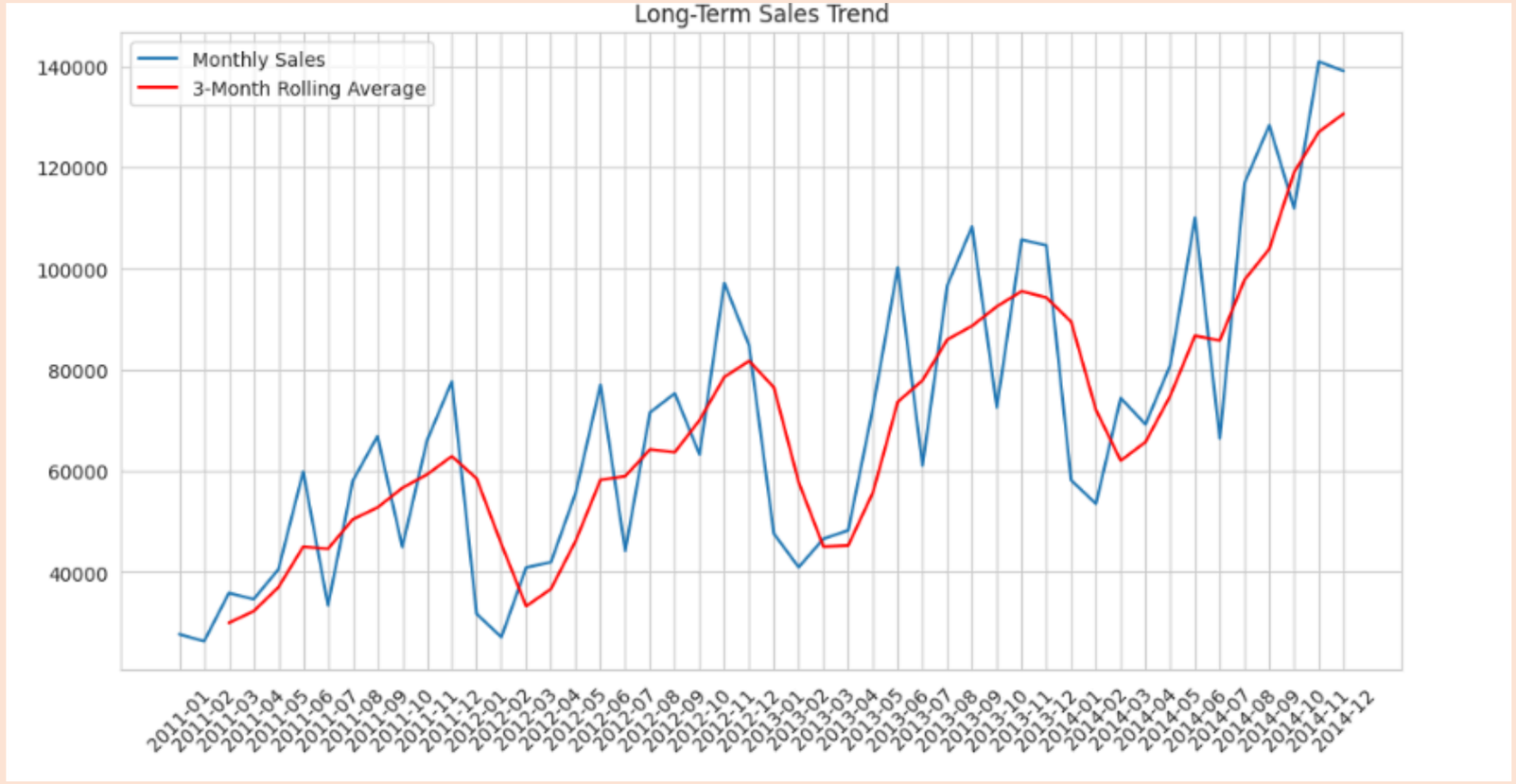
Data Analysis & Visualization

visualization of the relationships between sales, profit, discount, and quantity:



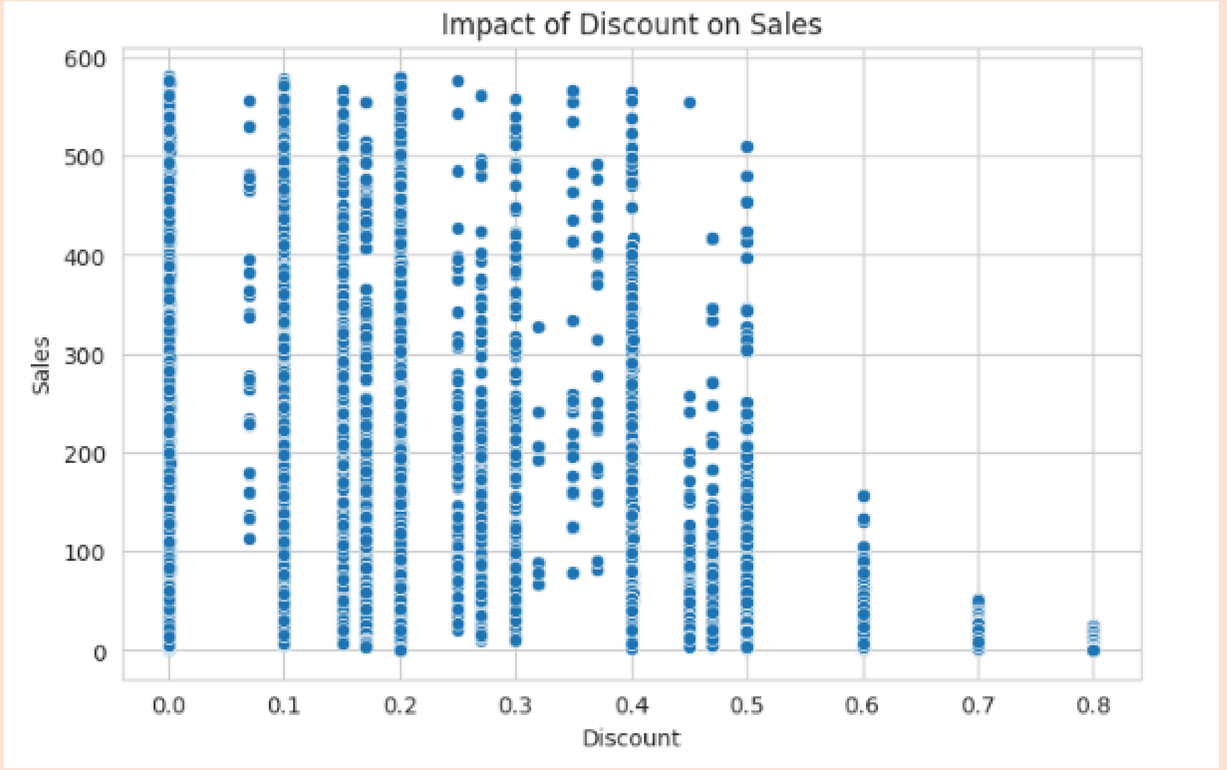
Data Analysis & Visualization

Identified seasonality and trends in monthly sales:



Data Analysis & Visualization

impact of promotions and discounts on sales

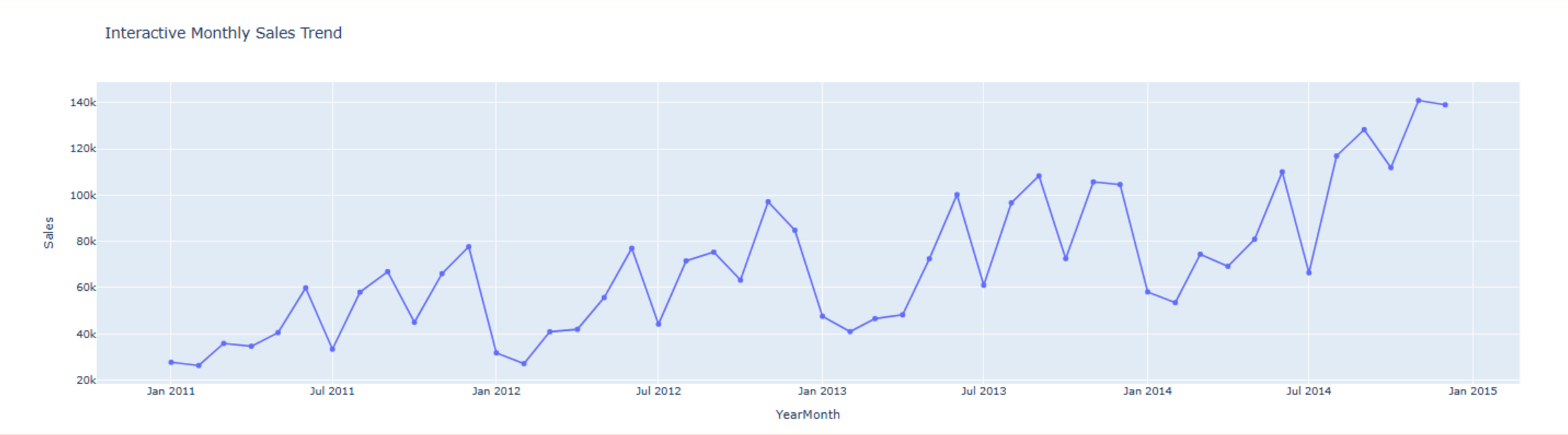


Data Analysis & Visualization

Examined profit vs. shipping cost relationships

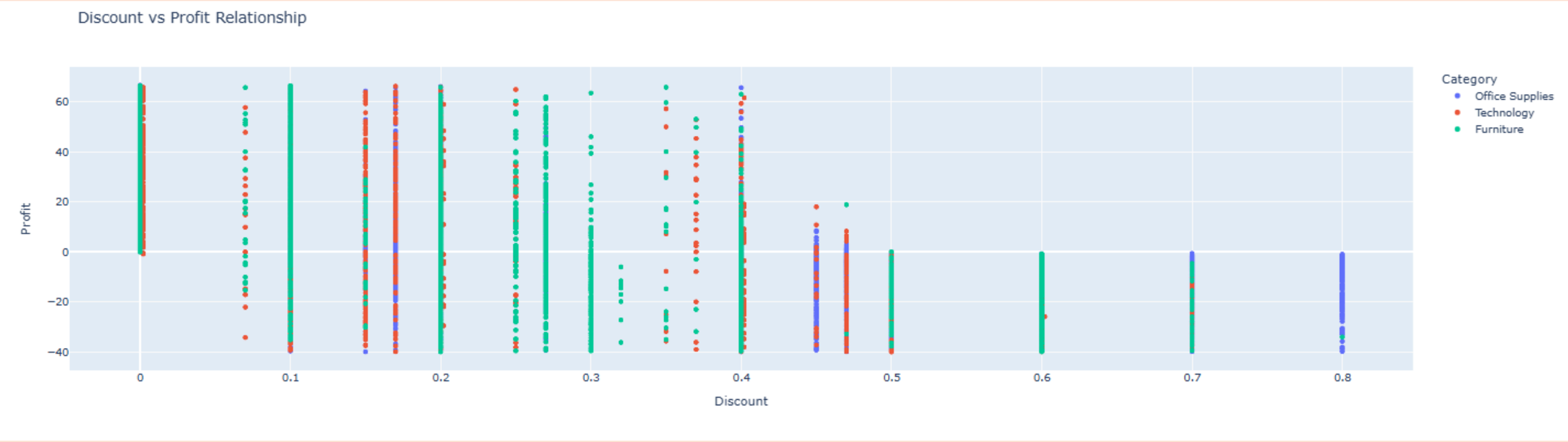
Data Analysis & Visualization

Created interactive visualizations for dashboards



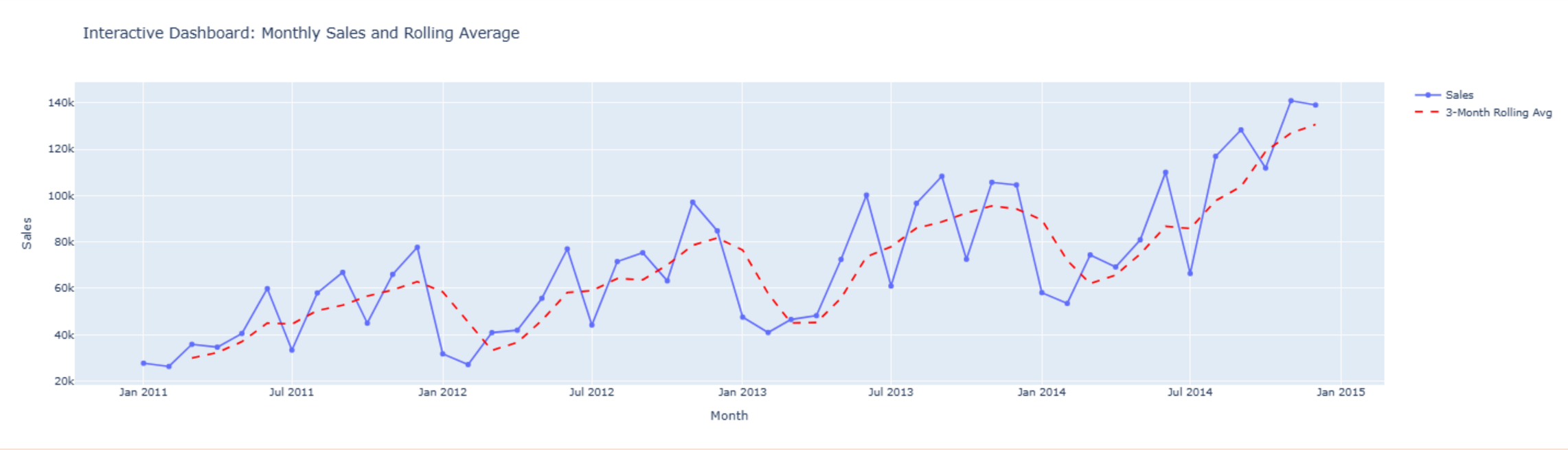
Data Analysis & Visualization

Created interactive visualizations for dashboards



Data Analysis & Visualization

Created interactive visualizations for dashboards



Model training and testing

Machine Learning Models Used:

- linear Regression
- Random Forest Regressor
- Gradient Boosting Regressor
- XGBoost Regressor
- Hyperparameter Tuning (GridSearchCV) for XGBoost

```
models = {
    "Linear Regression": LinearRegression(),
    "Random Forest": RandomForestRegressor(random_state=30),
    "Gradient Boosting": GradientBoostingRegressor(random_state=30),
    "XGBoost": XGBRegressor(random_state=30, verbosity=0)
}
```

Model training and testing

We chose these models because they cover different learning behaviors:

- Linear Regression → baseline linear model
- Random Forest → handles non-linear patterns
- Gradient Boosting → improves accuracy by learning errors
- XGBoost → modern optimized booster with strong performance

Model training and testing

This combination allows us to:

- Compare linear vs. non-linear methods
- Compare bagging (Random Forest) vs. boosting (GB, XGBoost)
- Identify the strongest model for prediction
- Detect overfitting or bias in each model

Model training and testing

Model Performance results Comparison:

```
Linear Regression:
  Train RMSE: 289.0485 | Test RMSE: 288.0933
  Train MAE:  191.2236 | Test MAE:  191.2931
  Train R²:   0.5674   | Test R²:   0.5667
--------------------------------------------------
Random Forest:
  Train RMSE: 72.6171 | Test RMSE: 192.5864
  Train MAE:  39.7844 | Test MAE:  106.2795
  Train R²:   0.9727  | Test R²:   0.8064
--------------------------------------------------
Gradient Boosting:
  Train RMSE: 202.6239 | Test RMSE: 206.0036
  Train MAE:  122.0300 | Test MAE:  123.5122
  Train R²:   0.7874   | Test R²:   0.7784
--------------------------------------------------
XGBoost:
  Train RMSE: 149.8635 | Test RMSE: 194.1039
  Train MAE:  85.9152 | Test MAE:  108.5824
  Train R²:   0.8837  | Test R²:   0.8033
--------------------------------------------------
```

Model training and testing

Tuned XGBoost Results:

GridSearchCV

```
Best parameters: {
            'colsample_bytree': 0.8,
            'gamma': 0.1,
            'learning_rate': 0.05,
            'max_depth': 7,
            'n_estimators': 300,
            'reg_alpha': 1,
            'reg_lambda': 2,
            'subsample': 1.0
}
```

```
Fitting 5 folds for each of 2916 candidates, totalling 14580 fits
Best parameters: {'colsample_bytree': 0.8, 'gamma': 0.1, 'learning_rate': 0.05, 'max_depth': 7, 'n_estimators': 300, 'reg_alpha': 1, 'reg_lambda': 2, 'subsample': 1.0}
Tuned XGBoost RMSE: 187.07
Tuned XGBoost MAE: 103.56
Tuned XGBoost R²: 0.8173
```

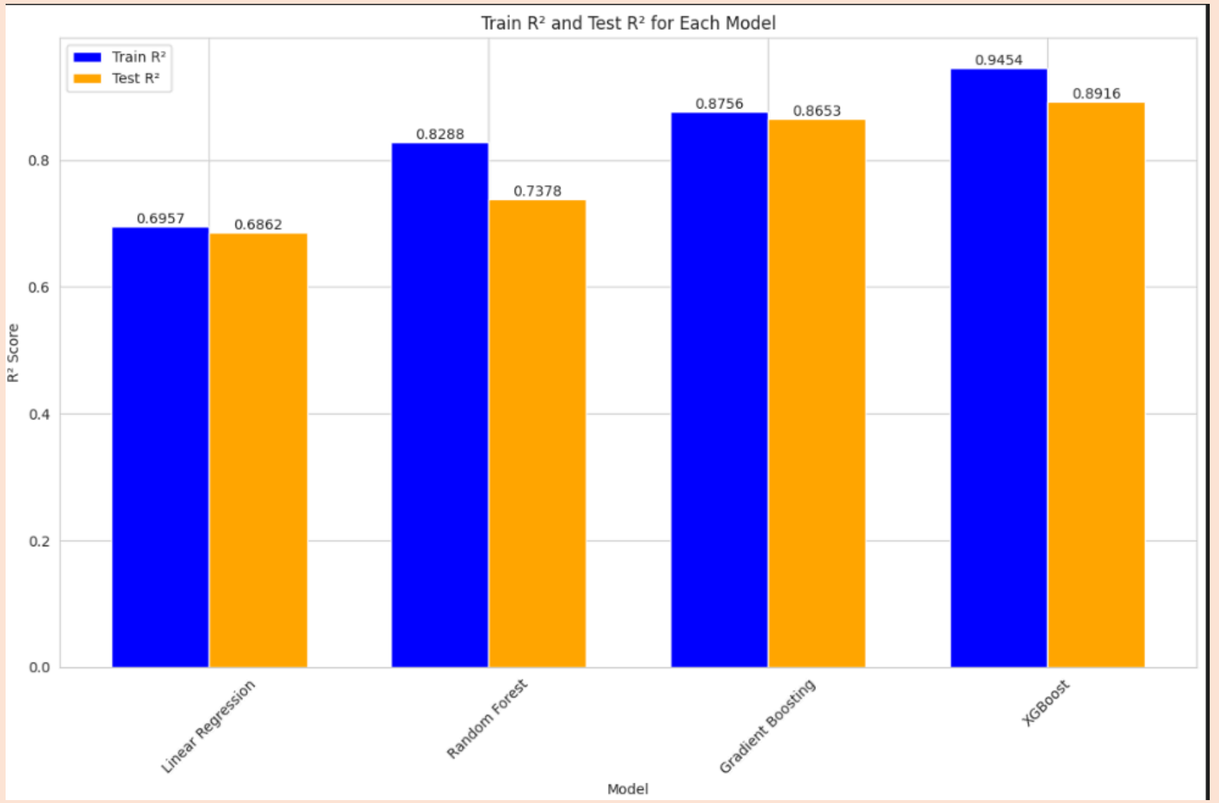
Model training and testing

Tuned XGBoost Results:

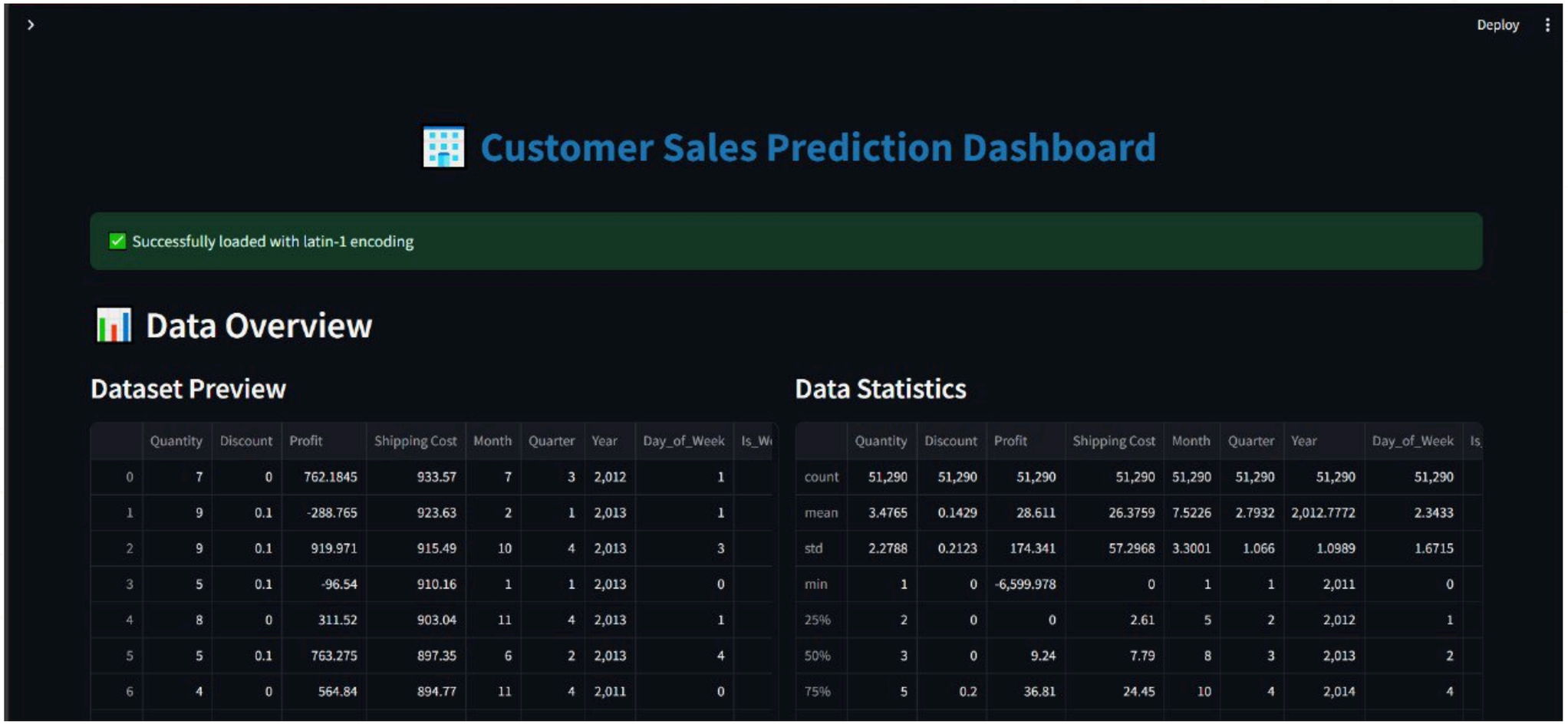
```
 Tuned XGBoost Performance:
RMSE: 187.5318
MAE:  104.4324
R²:   0.8228
```

Model training and testing

Visualization of the model results:



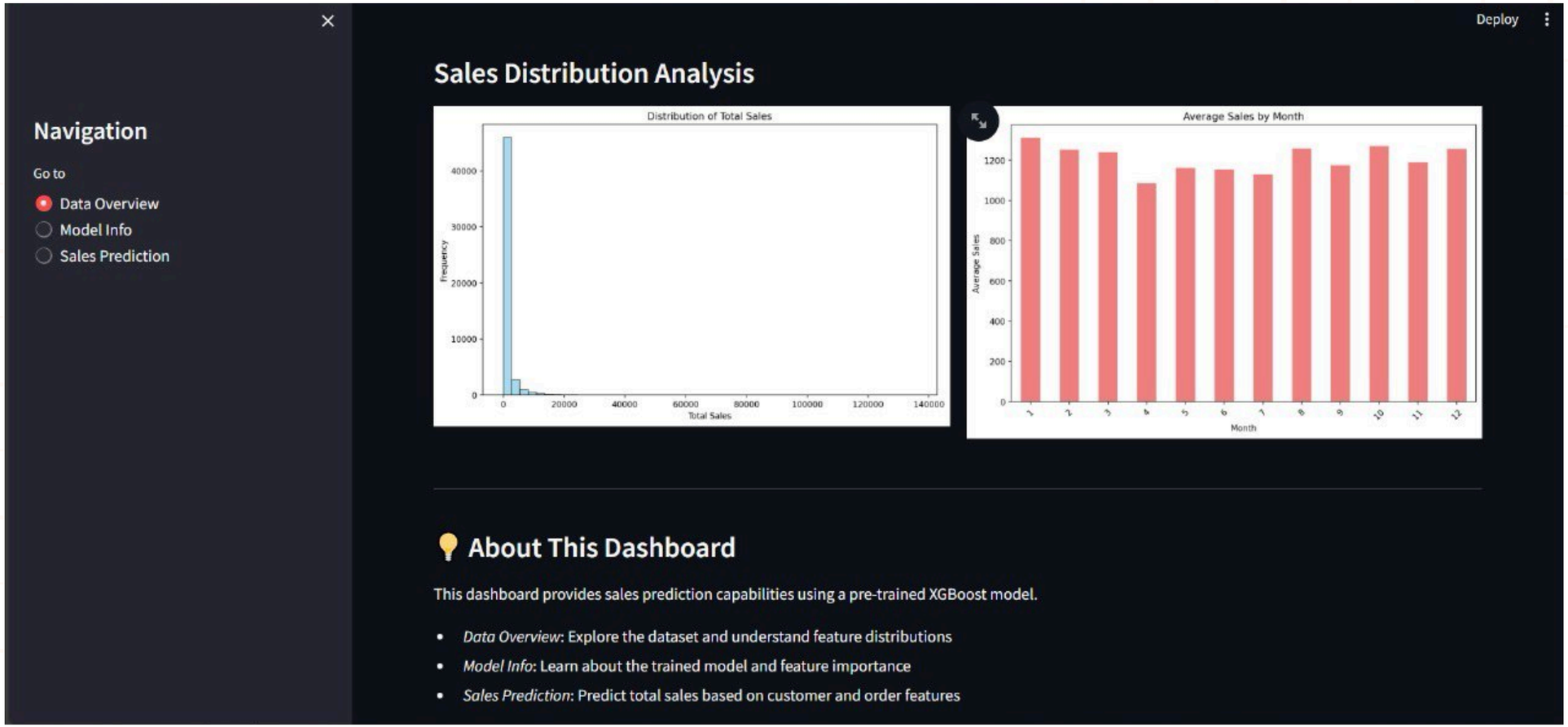
Model deployment



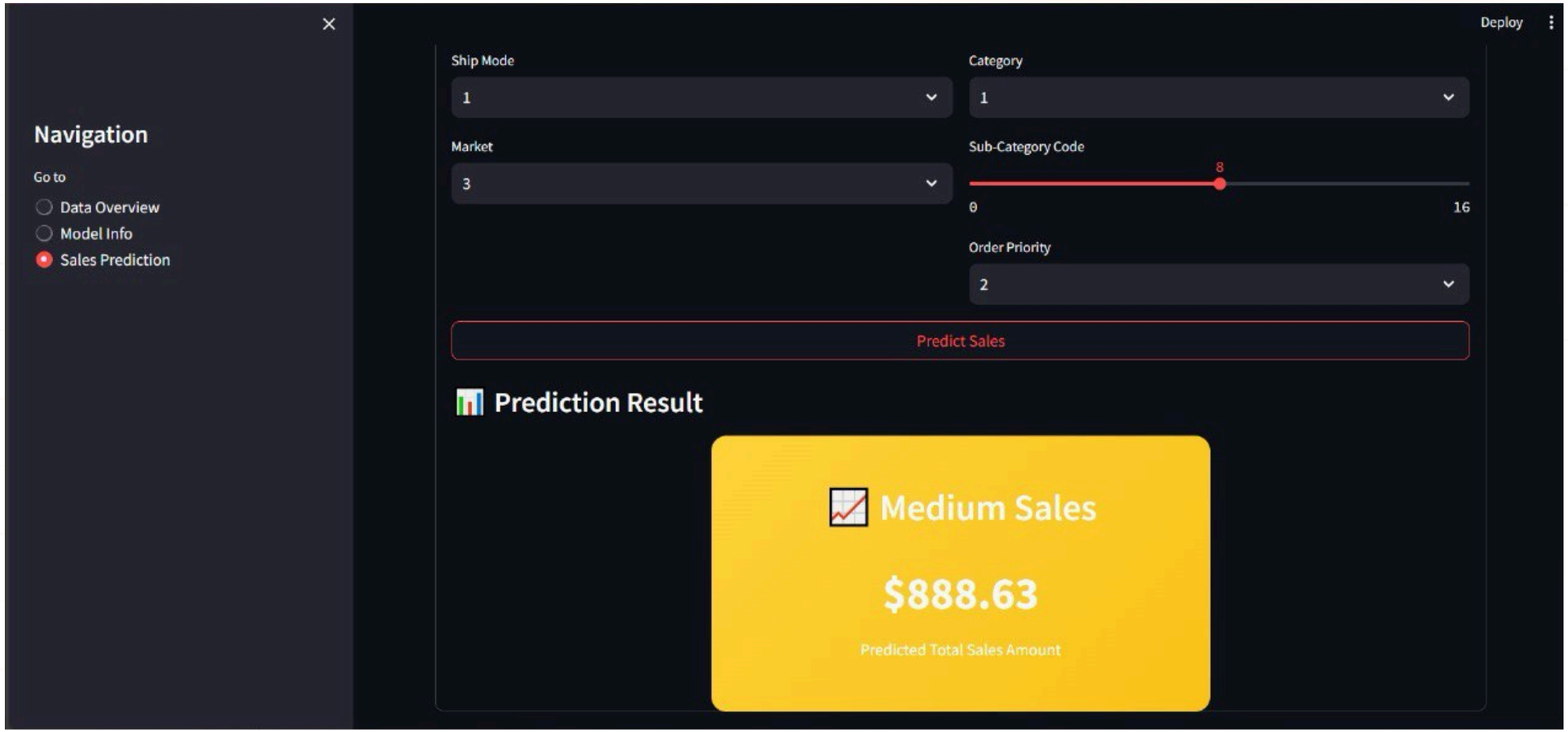
	Quantity	Discount	Profit	Shipping Cost	Month	Quarter	Year	Day_of_Week	Is_W
0	7	0	762.1845	933.57	7	3	2,012	1	
1	9	0.1	-288.765	923.63	2	1	2,013	1	
2	9	0.1	919.971	915.49	10	4	2,013	3	
3	5	0.1	-96.54	910.16	1	1	2,013	0	
4	8	0	311.52	903.04	11	4	2,013	1	
5	5	0.1	763.275	897.35	6	2	2,013	4	
6	4	0	564.84	894.77	11	4	2,011	0	

	Quantity	Discount	Profit	Shipping Cost	Month	Quarter	Year	Day_of_Week	Is
count	51,290	51,290	51,290	51,290	51,290	51,290	51,290	51,290	
mean	3.4765	0.1429	28.611	26.3759	7.5226	2.7932	2,012.7772	2.3433	
std	2.2788	0.2123	174.341	57.2968	3.3001	1.066	1.0989	1.6715	
min	1	0	-6,599.978	0	1	1	2,011	0	
25%	2	0	0	2.61	5	2	2,012	1	
50%	3	0	9.24	7.79	8	3	2,013	2	
75%	5	0.2	36.81	24.45	10	4	2,014	4	

Model deployment



Model deployment



Conclusion

In conclusion, we cleaned the data, chose the important features, trained our model, and got good results. The model learned the patterns in the dataset and can now make useful predictions.

With more data and more tuning, the model can become even better. This project helped us understand the full ML pipeline from start to end.


THANK YOU
FOR YOUR
PATIENCE